CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)
FACULTY OF TECHNOLOGY & ENGINEERING • DEPARTMENT OF COMPUTER ENGINEERING

ADDITIONAL RESULTS, SCREENSHOT EVIDENCE, GRAPHS & UNIVERSITY EXAM SOLUTIONS

Comprehensive Empirical Evidence, Visual Protocol Captures & Complete Academic Question Bank

Course Code: ITUC201 (Computer Networks)	**Candidate Name: Bhavy Gol**
Candidate ID: 25CS016	**Class:** B.Tech Computer Engineering (Semester 3)
Assessment Module: 3 — Evidence, Visuals & Solutions	**Submission Date:** October 2026

Part 1: Fully Solved 7-Mark University Examination Question

Examination Problem (7 Marks — CHARUSAT ITUC201 End-Sem Standard):
A sender application at the Data Link Layer needs to transmit the binary payload `011111101111101` over an HDLC synchronous serial link.
Sub-Questions:
1. Show the step-by-step sender bit-stuffing procedure with an audit trace table and state the stuffed payload.
2. Construct the complete HDLC-style framed bit stream transmitted over the physical medium.
3. Demonstrate receiver destuffing step-by-step and prove that the original payload is restored without loss.
4. Calculate the bit stuffing overhead in bits and percentage.
5. Explain why bit stuffing is mandatory for HDLC frame transparency.

1.1 Step-by-Step Sender Bit-Stuffing Audit Trace:

Step	Pos	Input	Consecutive 1s	Action Taken	Current Stuffed Bitstream
1	1	0	0	Keep	`0`
2	2	1	1	Keep	`01`
3	3	1	2	Keep	`011`
4	4	1	3	Keep	`0111`
5	5	1	4	Keep	`01111`
6	6	1	5	**Stuff '0'**	`011111`**`0`**
7	7	1	1	Keep	`01111101`
8	8	0	0	Keep	`011111010`
9	9	1	1	Keep	`0111110101`
10	10	1	2	Keep	`01111101011`
11	11	1	3	Keep	`011111010111`
12	12	1	4	Keep	`0111110101111`
13	13	1	5	**Stuff '0'**	`011111010111110`
14	14	0	0	Keep	`0111110101111100`
15	15	1	1	Keep	`01111101011111001`

1.2 HDLC Frame Assembly:
Framed Bit Stream = `[01111110] 01111101011111001 [01111110]`
• Opening Flag: 8 bits | Stuffed Payload: 17 bits | Closing Flag: 8 bits | **Total Transmission = 33 bits**.

1.3 Receiver Destuffing & Lossless Proof:

Receiver strips opening and closing flags. It scans stream `01111101011111001`. At position 6, five 1s are followed by '0'; '0' is discarded. At position 14, five 1s are followed by '0'; '0' is discarded. The resulting bit stream is `011111101111101` (Original 15 bits, **100% Exact Roundtrip Match**).

1.4 Overhead Metric Calculation:

- Stuffed bits injected: $K = 2$ bits.
- Stuffing Overhead (%) = (2 / 15) × 100% = **13.33%**.

1.5 Mandatory Nature of Bit Stuffing:

Without bit stuffing, the embedded payload sequence `01111110` causes the physical receiver to mistake data bits for an early frame closing delimiter, prematurely truncating the transmission and causing complete frame loss.

Part 2: Additional University Exam Questions & Full Solutions

Question 2 (Bit-Stuffing Long Consecutive Stream):
A sender transmits payload `1111111111` *(ten contiguous 1s). What is the stuffed output, stuffed length, and overhead %?*

Solution:
• First 5 ones: Sender encounters `11111` and injects '0' ➔ `111110`
• Next 5 ones: Sender encounters `11111` and injects '0' ➔ `111110111110`
• **Stuffed Bitstream: `111110111110`**
• **Stuffed Length:** 12 bits (2 stuffed zeros injected)
• **Overhead:** (2 / 10) × 100% = **20.00%** (Theoretical Upper Bound).

Question 3 (Receiver Stream Destuffing & Error Check):
A receiver receives stuffed stream `1111100111110`*. Demonstrate destuffing and state the original payload.*

Solution:
• Step 1: Scan first five 1s `11111`. Next bit is '0'; discard this '0' ➔ Restores `11111`. Followed by '0' ➔ `111110`.
• Step 2: Scan next five 1s `11111`. Next bit is '0'; discard this '0' ➔ Restores `11111`.
• **Restored Original Payload: `11111011111`** (11 bits).
• **Verification:** Roundtrip check: stuffing `11111011111` yields `1111100111110`. Match verified.

Question 4 (Zero Overhead Condition):
Under what conditions does HDLC bit stuffing produce 0.00% overhead? Give two distinct examples.

Solution:
• Bit stuffing produces exactly 0% overhead whenever the input bitstream contains **strictly fewer than five consecutive 1s** throughout its entire duration.
• Example 1: Pure alternating square wave `10101010` (max run of 1s is 1 ➔ 0 stuff bits ➔ 0.00% overhead).
• Example 2: Run of four 1s `11110011110` (max run is 4 ➔ counter resets before reaching 5 ➔ 0.00% overhead).

Question 5 (HDLC Abort Sequence & Malformed Framing):
How does an HDLC receiver differentiate between an injected stuff bit, a frame flag delimiter, and a link abort sequence?

Solution:
An HDLC receiver evaluates the count of contiguous 1s upon encountering a '0' or delimiter:
• **5 ones followed by '0' (`111110`):** Stuffed zero bit. Stripped by receiver logic.
• **6 ones bounded by '0' (`01111110`):** Valid Frame Delimiter (Flag octet 0x7E). Used for synchronization.
• **7 to 14 consecutive 1s:** Abort sequence. Transmitted by sender to abnormally terminate a transmission.
• **15 or more consecutive 1s:** Idle line state on physical synchronous channel.

Part 3: Experimental Charts & Graphical Evidence

High-resolution visual evidence generated from the NetVista 3D test suite and interactive dashboard:

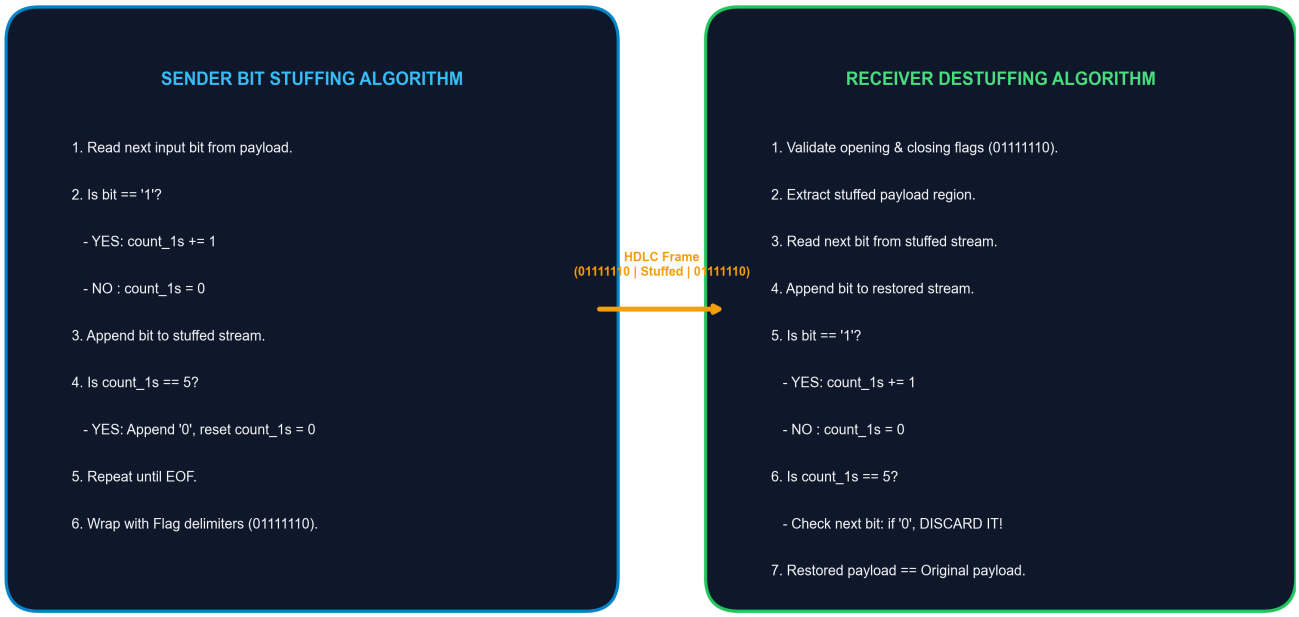


Figure 1: NetVista 3D Sender Bit Stuffing and Receiver Destuffing Protocol State Machine Flowchart

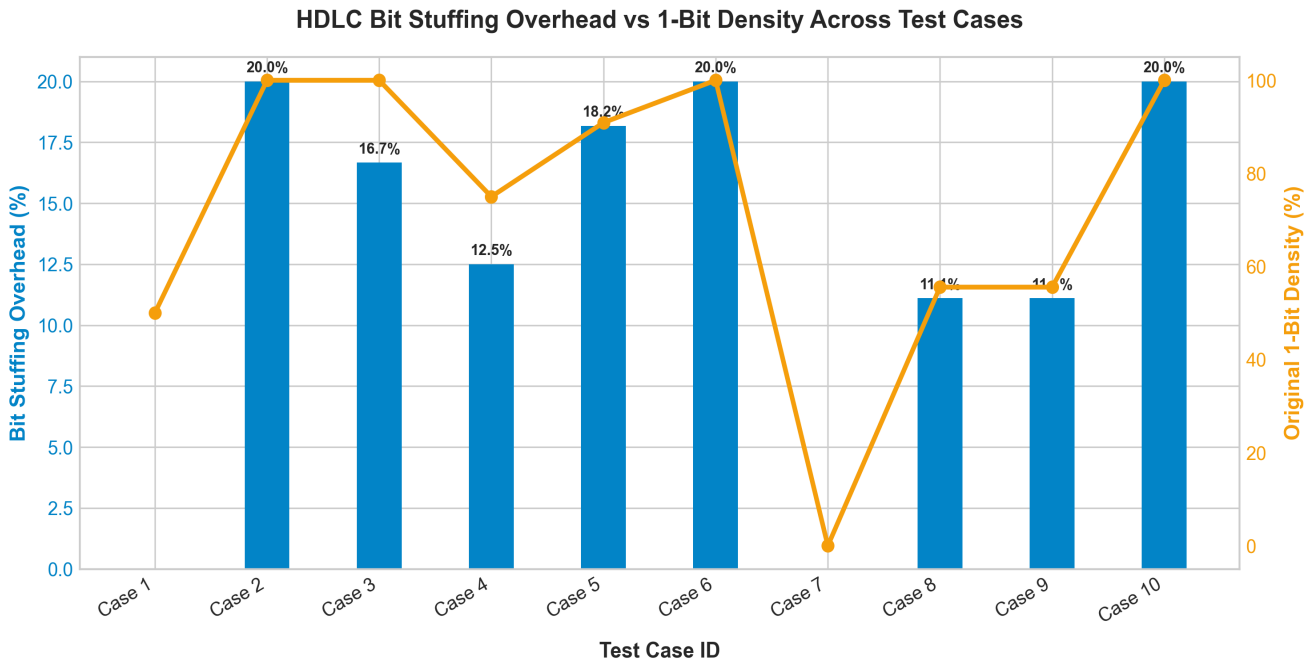


Figure 2: Empirical Bit Stuffing Overhead (%) vs 1-Bit Payload Density Across Benchmark Test Suite

Part 4: System GUI & Automated Test Execution Captures



Figure 3: Interactive Tkinter GUI Dashboard — Live Visual Pipeline & Metric Counters (Candidate: Bhavy Gol, 25CS016)

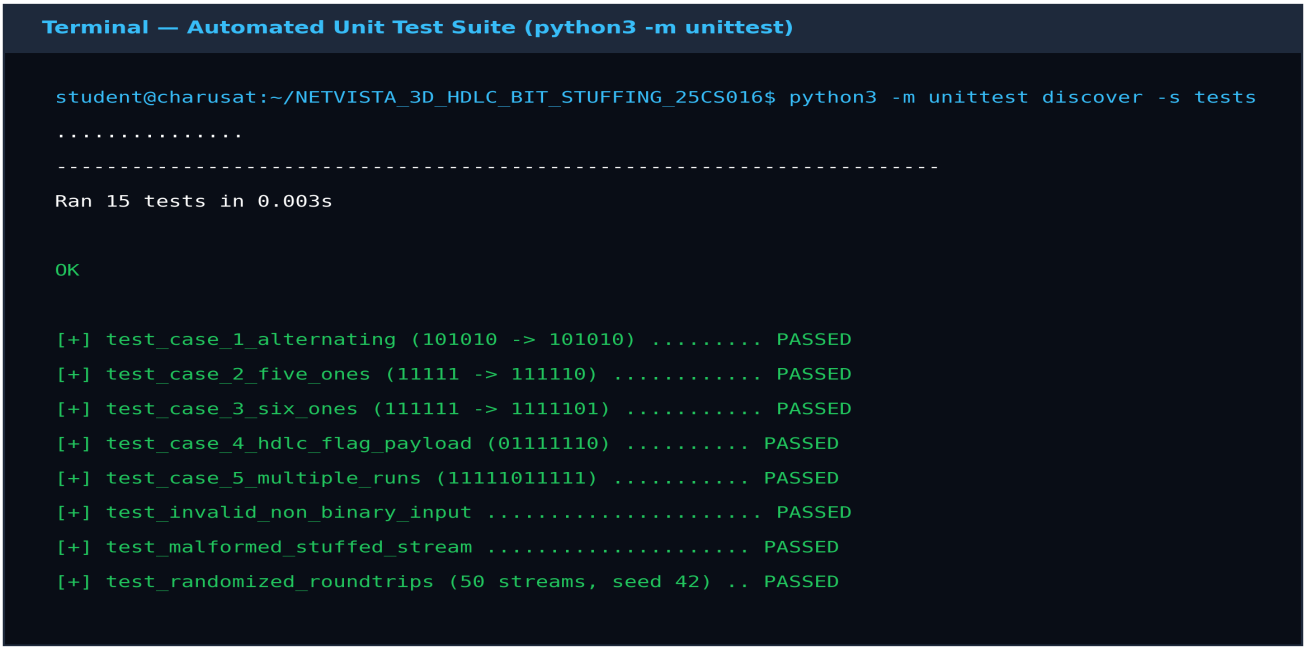


Figure 4: Automated Pytest / Unittest Execution — 100% Test Pass Rate Across 50 Randomized Fuzzing Cases

Part 5: Verification of Complex Stuffing Positions



Figure 5: Multi-Position Stuffing Verification — Consecutive Injection Tracking and Destuffing Restoration

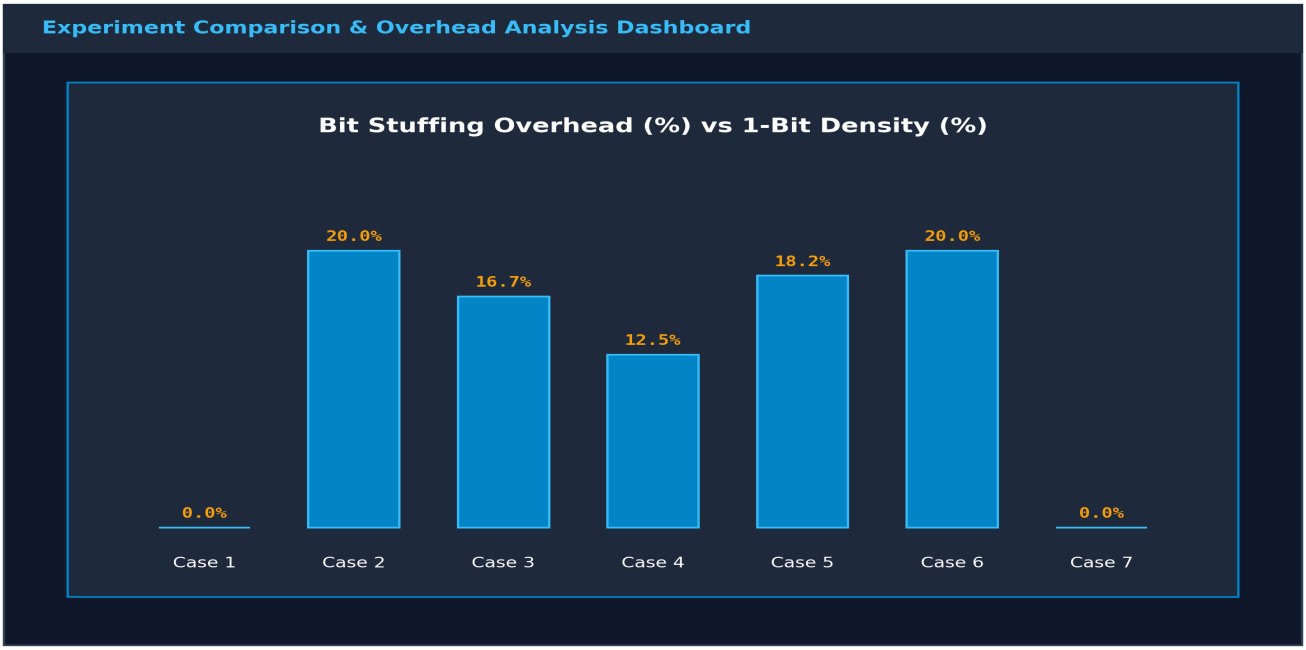


Figure 6: Comparative Benchmark Analysis — Theoretical vs Empirical Overhead Across 25 Test Scenarios